МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
КАФЕДРА САПР

Практическая работа №1

по дисциплине

«Объектно-ориентированное программирование»

Студент гр. 5352 _____ Решетников С.А

Преподаватель _____ Кулагин М.В.

Санкт-Петербург
2026

1. Задание

Разработать консольное приложение, которое моделирует работу небольшой службы доставки.

Пользователь должен иметь возможность создать заказ, выбрать способ доставки, назначить курьера, просмотреть список заказов и переводить заказ по допустимым статусам. В программе должны быть предусмотрены корректная обработка ошибочного ввода и понятные сообщения для пользователя.

2. Спецификация программы

Центральным элементом системы является класс `Order` (Заказ), который связывает между собой ключевые сущности предметной области. Он находится в отношениях агрегации с классом `Client` (у каждого заказа есть клиент) и с классом `Courier` (назначаемый курьер может временно отсутствовать или не требоваться вовсе), а также ссылается на стратегию расчета через интерфейс `DeliveryMethodInterface`. При этом заказ находится в жесткой связи композиции с классом `OrderItem` (список товаров принадлежит конкретному заказу и формирует его состав). Текущий статус заказа строго типизирован с помощью перечисления `OrderStatus`.

Абстрактный контракт `DeliveryMethodInterface` реализуется конкретными классами `StandartDelivery`, `ExpressDelivery` и `PickupDelivery`. Управлением списками данных и бизнес-правилами занимается сервисный класс `OrderService`, который делегирует персистентность (сохранение в JSON) классу `StorageService`. Завершает структуру пользовательский интерфейс `ConsoleUI`, связывающий консольный ввод с логикой сервисов.

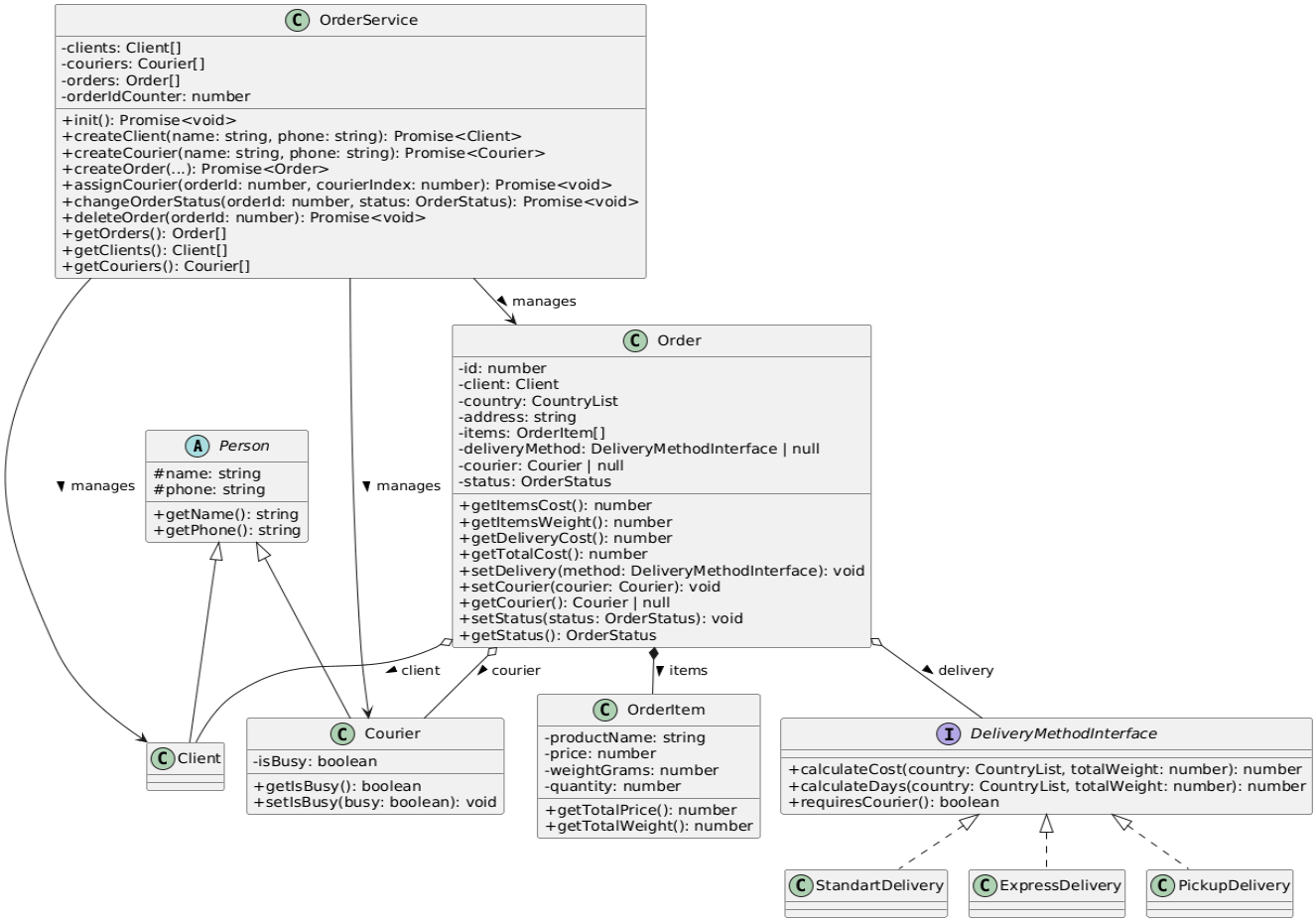


Рис. 1 - Диаграмма классов

3. Описание интерфейса пользователя программы

Пользовательский интерфейс программы реализован в виде интерактивного консольного приложения командной строки (CLI), разработанного с использованием встроенного асинхронного модуля Node.js `readline/promises`. Такой подход позволяет организовать пошаговое взаимодействие с пользователем в удобном линейном стиле через конструкции `async/await`, одновременно предотвращая блокировку основного потока выполнения приложения. Интерфейс спроектирован с упором на эргономику и читаемость в условиях терминала, минимизируя визуальный шум и предоставляя четкую обратную связь на каждом этапе работы.

При запуске системы происходит асинхронная инициализация всех необходимых сервисов и загрузка сохраненных данных, после чего перед пользователем открывается главное меню (рис. 2). Оно предоставляет доступ ко всем ключевым сценариям взаимодействия: от регистрации клиентов и курьеров до формирования сложных заказов, назначения курьеров, смены их статусов и просмотра сводной отчетности. Главное меню выступает в роли единой точки навигации, возвращая пользователя на основной экран после завершения любой выбранной операции.

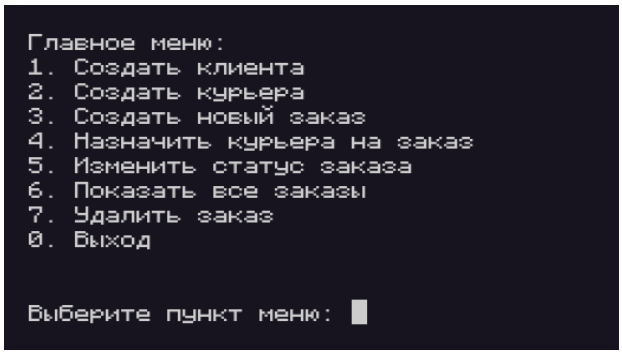


Рис. 2 — Главное меню программы

На рис. 3 показан пошаговый процесс создания заказа на основе введёных пользователем данных. После окончания создания заказа он сохраняется в json файл.

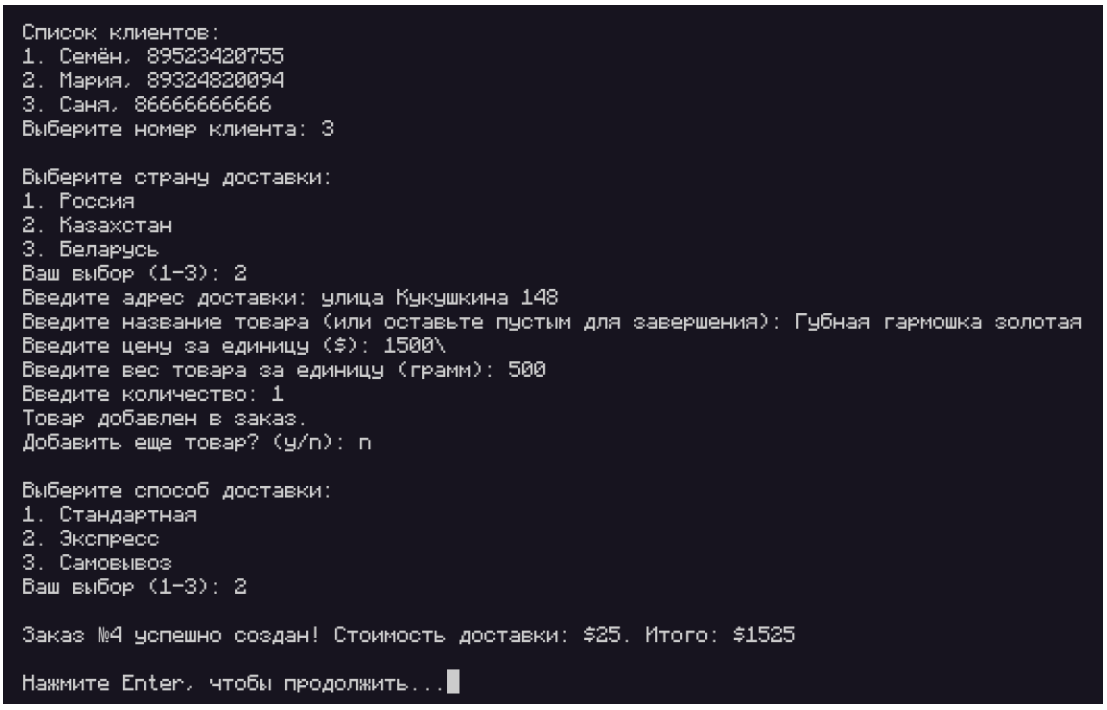


Рис. 3 — Пошаговый процесс создания заказа

После создания заказа мы можем также поменять его статус (рис. 5) и назначить подходящего курьера из выпадающего списка созданных курьеров (рис. 4).

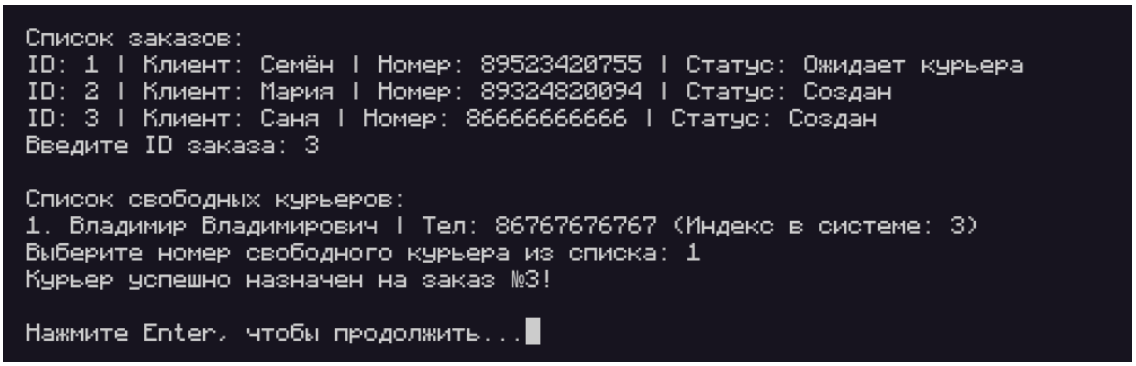


Рис. 4 — Назначение курьера

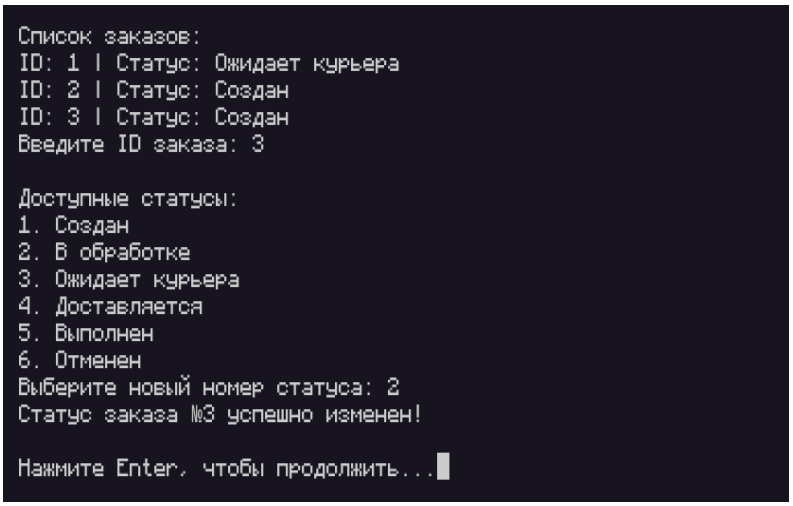


Рис. 5 — Изменение статуса заказа

Для просмотра детального списка всех актуальных заказов в программе предусмотрено отдельное меню, которое выводит список всех заказов (рис. 6) и содержащейся в них информации: ID заказа, клиент, назначенный курьер, адрес, статус, список товаров и их стоимость, стоимость доставки, срок доставки.

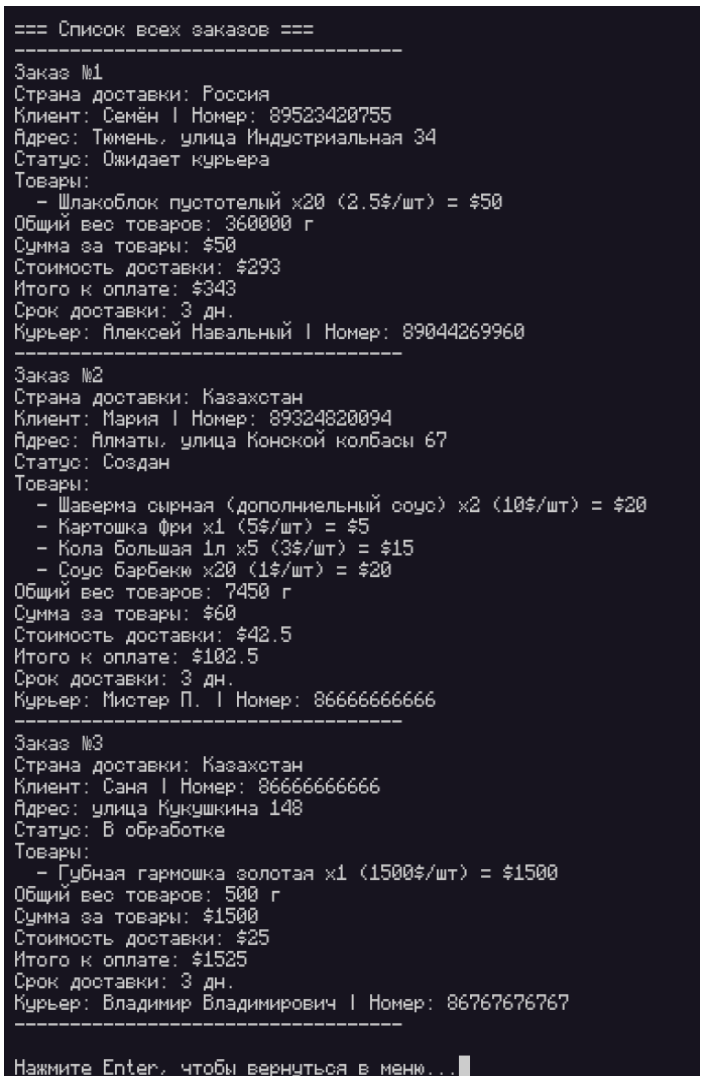


Рис. 6 — Список всех заказов

4. Текст программы

index.ts

```
import { ConsoleUI } from "./ui/CUI.js";
const ui = new ConsoleUI();
ui.start().catch((err) => {
    console.error("Критическая ошибка приложения:", err);
    process.exit(1);
});
```

Person.ts

```
export abstract class Person {
    constructor (
```

```
        public readonly name: string,
        public readonly phone: string
    ) {}
}
```

Client.ts

```
import { Person } from "./Person"

export class Client extends Person {
    constructor(name: string, phone: string) {
        super(name, phone);
    }
}
```

Courier.ts

```
import { Person } from "./Person"

export class Courier extends Person {
    private isBusy: boolean = false;

    constructor (name: string, phone: string) {
        super(name, phone);
    }

    public getIsBusy(): boolean {
        return this.isBusy;
    }

    public setIsBusy(busy: boolean): void {
        this.isBusy = busy;
    }
}
```

Order.ts

```
import { Client } from "./Client";
import { Courier } from "./Courier";
import { OrderItem } from "./OrderItem";
import { OrderStatus } from "./OrderStatus";
import { CountryList } from "./CountryList";
import { DeliveryMethodInterface } from "../delivery/DeliveryMethodInterface"

export class Order {
    private status: OrderStatus = OrderStatus.CREATED;
```

```
    private delivery?: DeliveryMethodInterface;
    private courier?: Courier;

    constructor (
        public readonly id: number,
        public readonly client: Client,
        public readonly country: CountryList,
        public readonly address: string,
        public readonly items: OrderItem[]
    ) {}

    public getStatus(): OrderStatus {
        return this.status;
    }

    public getDelivery(): DeliveryMethodInterface | undefined {
        return this.delivery;
    }

    public getCourier(): Courier | undefined {
        return this.courier;
    }

    public setStatus(newStatus: OrderStatus): void {
        this.status = newStatus;
    }

    public setDelivery(newDelivery: DeliveryMethodInterface): void {
        this.delivery = newDelivery;
    }

    public setCourier(newCourier: Courier): void {
        this.courier = newCourier;
    }

    public getItemsCost(): number {
        return this.items.reduce((sum, item) => sum + item.getTotalPrice(),
0)
    }

    public getItemsWeight(): number {
        return this.items.reduce((sum, item) => sum + item.getTotalWeight(),
0)
    }
}
```

OrderItem.ts

```
export class OrderItem {
```

```
    constructor (
        public readonly productName: string,
        public readonly price: number,
        public readonly weight: number,
        public readonly quantity: number
    ) {}

    public getTotalPrice(): number {
        return this.price * this.quantity;
    }

    public getTotalWeight(): number {
        return this.weight * this.quantity;
    }
}
```

OrderStatus.ts

```
export enum OrderStatus {
    CREATED = "Создан",
    PROCESSING = "В обработке",
    AWAITING_COURIER = "Ожидает курьера",
    DELIVERING = "Доставляется",
    COMPLETED = "Выполнен",
    CANCELLED = "Отменен"
}
```

DeliveryMethodInterface.ts

```
import { CountryList } from "../models/CountryList";

export interface DeliveryMethodInterface {
    calculateCost(country: CountryList, weight: number): number;
    calculateDays(country: CountryList, weight: number): number;
    requiresCourier(): boolean;
}
```

ExpressDelivery.ts

```
iimport { DeliveryMethodInterface } from "./DeliveryMethodInterface";
import { CountryList } from "../models/CountryList";

export class ExpressDelivery implements DeliveryMethodInterface {
    public calculateCost(country: CountryList, weight: number): number {
        const deliveryFee: number = 2.5;
        const baseCost: number = 5;

        let countryCost: number = 0;
```

```
        switch (country) {
        case CountryList.RUS: countryCost = 10; break
        case CountryList.KAZ: countryCost = 20; break
        case CountryList.BLR: countryCost = 30; break
        }
        const weightCost: number = deliveryFee * (Math.trunc(weight / 1000));

        return baseCost + countryCost + weightCost;
    }

    public calculateDays(country: CountryList): number {
        if (country == CountryList.RUS) return 1;
        else if (country == CountryList.KAZ) return 3;
        else if (country == CountryList.BLR) return 5;
        else return 0;
    }

    public requiresCourier(): boolean {
        return true;
    }
}
```

StandartDelivery.ts

```
import { DeliveryMethodInterface } from "./DeliveryMethodInterface";
import { CountryList } from "../models/CountryList";

export class StandartDelivery implements DeliveryMethodInterface {
    public calculateCost(country: CountryList, weight: number): number {
        const deliveryFee: number = 0.8;
        const baseCost: number = 3;

        let countryCost: number = 0;
        if (country == CountryList.RUS) countryCost = 2;
        else if (country == CountryList.KAZ) countryCost = 10;
        else if (country == CountryList.BLR) countryCost = 15;

        const weightCost: number = deliveryFee * (Math.trunc(weight / 1000));

        return baseCost + countryCost + weightCost;
    }

    public calculateDays(country: CountryList): number {
        if (country == CountryList.RUS) return 3;
        else if (country == CountryList.KAZ) return 7;
        else if (country == CountryList.BLR) return 10;
        else return 0;
    }
```

```
    public requiresCourier(): boolean {
        return true;
    }
}
```

PickupDelivery.ts

```
import { DeliveryMethodInterface } from "./DeliveryMethodInterface";

export class PickupDelivery implements DeliveryMethodInterface {
    public calculateCost(): number {
        return 0; /* Самовывоз бесплатно */
    }

    public calculateDays(): number {
        return 0;
    }

    public requiresCourier(): boolean {
        return false;
    }
}
```

OrderService.ts

```
import { Client } from "../models/Client.js";
import { Courier } from "../models/Courier.js";
import { Order } from "../models/Order.js";
import { OrderItem } from "../models/OrderItem.js";
import { OrderStatus } from "../models/OrderStatus.js";
import { DeliveryMethodInterface } from
"../delivery/DeliveryMethodInterface.js";
import { StorageService } from "./StorageService.js";
import { CountryList } from "../models/CountryList.js";

export class OrderService {
    private storage = new StorageService();

    private clients: Client[] = [];
    private couriers: Courier[] = [];
    private orders: Order[] = [];
    private orderIdCounter: number = 1;

    // Загрузка данных при старте
    public async init(): Promise<void> {
        const loaded = await this.storage.loadData();
        this.clients = loaded.clients;
```

```
        this.couriers = loaded.couriers;
        this.orders = loaded.orders;
        this.orderIdCounter = loaded.orderIdCounter;
    }

    // Сохранение текущего состояния в JSON
    private async persist(): Promise<void> {
        await this.storage.saveData(this.clients, this.couriers, this.orders,
this.orderIdCounter);
    }

    public getClients(): Client[] {
        return this.clients;
    }

    public getCouriers(): Courier[] {
        return this.couriers;
    }

    public getOrders(): Order[] {
        return this.orders;
    }

    // Бизнес-логика создания клиента
    public async createClient(name: string, phone: string): Promise<Client> {
        if (!name.trim()) throw new Error("Имя клиента не может быть
пустым!");
        if (!phone.trim()) throw new Error("Номер телефона клиента не может
быть пустым!");
        const client = new Client(name.trim(), phone.trim());
        this.clients.push(client);
        await this.persist();
        return client;
    }

    // Бизнес-логика создания курьера
    public async createCourier(name: string, phone: string): Promise<Courier>
{
        if (!name.trim()) throw new Error("Имя курьера не может быть
пустым!");
        if (!phone.trim()) throw new Error("Номер телефона курьера не может
быть пустым!");
        const courier = new Courier(name.trim(), phone.trim());
        this.couriers.push(courier);
        await this.persist();
        return courier;
    }

    // Бизнес-логика создания заказа
    public async createOrder(
```

```
        clientIndex: number,
        address: string,
        country: CountryList,
        items: OrderItem[],
        deliveryMethod: DeliveryMethodInterface
    ): Promise<Order> {
        if (this.clients.length === 0) {
            throw new Error("Сначала создайте хотя бы одного клиента!");
        }
        if (!this.clients[clientIndex]) {
            throw new Error("Неверный номер клиента!");
        }
        if (!address.trim()) {
            throw new Error("Адрес доставки не может быть пустым!");
        }
        if (!country.trim()) {
            throw new Error("Страна доставки не может быть пустой!");
        }
        if (items.length === 0) {
            throw new Error("В заказе должен быть хотя бы один товар!");
        }

        const client = this.clients[clientIndex];
        const order = new Order(this.orderIdCounter++, client, country,
address.trim(), items);
        order.setDelivery(deliveryMethod);

        this.orders.push(order);
        await this.persist();
        return order;
    }

    // Назначение курьера на заказ
    public async assignCourier(orderId: number, courierIndex: number):
Promise<void> {
        const order = this.orders.find(o => o.id === orderId);
        if (!order) throw new Error("Заказ с таким ID не найден!");

        const delivery = order.getDelivery();
        if (delivery && !delivery.requiresCourier()) {
            throw new Error("Для данного способа доставки курьер не
требуется!");
        }

        if (!this.couriers[courierIndex]) {
            throw new Error("Неверный номер курьера!");
        }

        const courier = this.couriers[courierIndex];
```

```
        order.setCourier(courier);
        courier.setIsBusy(true);

        await this.persist();
    }

    // Изменение статуса заказа
    public async changeOrderStatus(orderId: number, newStatus: OrderStatus):
Promise<void> {
        const order = this.orders.find(o => o.id === orderId);
        if (!order) throw new Error("Заказ не найден!");

        order.setStatus(newStatus);
        await this.persist();
    }

    public async deleteOrder(orderId: number): Promise<void> {
        const index = this.orders.findIndex(o => o.id === orderId);
        if (index === -1) throw new Error("Заказ не найден!");

        const order = this.orders[index];

        const courier = order.getCourier();
        if (courier) {
            courier.setIsBusy(false);
        }

        this.orders.splice(index, 1);

        await this.persist();
    }
}
```

StorageService.ts

```
import * as fs from "fs/promises";
import * as path from "path";
import { Client } from "../models/Client.js";
import { Courier } from "../models/Courier.js";
import { Order } from "../models/Order.js";
import { OrderItem } from "../models/OrderItem.js";
import { OrderStatus } from "../models/OrderStatus.js";
import { DeliveryMethodInterface } from
"../delivery/DeliveryMethodInterface.js";
import { StandartDelivery } from "../delivery/StandartDelivery.js";
import { ExpressDelivery } from "../delivery/ExpressDelivery.js";
import { PickupDelivery } from "../delivery/PickupDelivery.js";
import { CountryList } from "../models/CountryList.js";

interface DatabaseSchema {
```

```
    clients: {
        name: string;
        phone: string;
    }[];
    couriers: {
        name: string;
        phone: string;
        isBusy: boolean;
    }[];
    orders: {
        id: number;
        clientName: string;
        clientPhone: string;
        country: CountryList;
        address: string;
        status: OrderStatus;
        items: { productName: string;
                price: number;
                weight: number;
                quantity: number;
        }[];
        deliveryType: "Standart" | "Express" | "Pickup";
        courierName?: string;
        courierPhone?: string;
    }[];
    orderIdCounter: number;
}

export class StorageService {
    private filePath: string;

    constructor(fileName: string = "src/data/database.json") {
        this.filePath = path.resolve(process.cwd(), fileName);
    }

    public async saveData(
        clients: Client[],
        couriers: Courier[],
        orders: Order[],
        orderIdCounter: number
    ): Promise<void> {
        const data: DatabaseSchema = {
            clients: clients.map(c => ({ name: c.name, phone: c.phone})),
            couriers: couriers.map(co => ({ name: co.name, phone: co.phone,
isBusy: co.getIsBusy() })),
            orders: orders.map(o => {
                let deliveryType: "Standart" | "Express" | "Pickup" =
"Standart";
                const method = o.getDelivery();
```

```
                if (method instanceof ExpressDelivery) deliveryType =
"Express";
                else if (method instanceof PickupDelivery) deliveryType =
"Pickup";

                return {
                    id: o.id,
                    clientName: o.client.name,
                    clientPhone: o.client.phone,
                    country: o.country,
                    address: o.address,
                    status: o.getStatus(),
                    items: o.items.map(i => ({
                        productName: i.productName,
                        price: i.price,
                        weight: i.weight,
                        quantity: i.quantity
                    })),
                    deliveryType,
                    courierName: o.getCourier()?.name,
                    courierPhone: o.getCourier()?.phone
                };
            }),
            orderIdCounter
        };

        await fs.writeFile(this.filePath, JSON.stringify(data, null, 2),
"utf-8");
    }

    public async loadData(): Promise<{
        clients: Client[];
        couriers: Courier[];
        orders: Order[];
        orderIdCounter: number;
    }> {
        try {
            const fileContent = await fs.readFile(this.filePath, "utf-8");
            const data: DatabaseSchema = JSON.parse(fileContent);

            const clients = data.clients.map(c => new Client(c.name,
c.phone));

            const couriers = data.couriers.map(co => {
                const courier = new Courier(co.name, co.phone);
                courier.setIsBusy(co.isBusy);
                return courier;
            });
```

```
            const orders = data.orders.map(oData => {
                const client = clients.find(c => (c.name === oData.clientName
&& c.phone === oData.clientPhone)) || new Client(oData.clientName,
oData.clientPhone);
                const items = oData.items.map(i => new
OrderItem(i.productName, i.price, i.weight, i.quantity));

                const order = new Order(oData.id, client, oData.country,
oData.address, items);
                order.setStatus(oData.status);

                let deliveryMethod: DeliveryMethodInterface;
                if (oData.deliveryType === "Express") deliveryMethod = new
ExpressDelivery();
                else if (oData.deliveryType === "Pickup") deliveryMethod =
new PickupDelivery();
                else deliveryMethod = new StandartDelivery();

                order.setDelivery(deliveryMethod);

                if (oData.courierName) {
                    const foundCourier = couriers.find(co => (co.name ===
oData.courierName && co.phone === oData.courierPhone));
                    if (foundCourier) {
                        order.setCourier(foundCourier);
                    }
                }

                return order;
            });

            return {
                clients,
                couriers,
                orders,
                orderIdCounter: data.orderIdCounter || (orders.length > 0 ?
Math.max(...orders.map(o => o.id)) + 1 : 1)
            };
        } catch (error) {
            return {
                clients: [],
                couriers: [],
                orders: [],
                orderIdCounter: 1
            };
        }
    }
}
```

CUI.ts

```
import * as readline from "readline/promises";
import { stdin as input, stdout as output } from "process";
import { OrderService } from "../services/OrderService.js";
import { OrderItem } from "../models/OrderItem.js";
import { OrderStatus } from "../models/OrderStatus.js";
import { StandartDelivery } from "../delivery/StandartDelivery.js";
import { ExpressDelivery } from "../delivery/ExpressDelivery.js";
import { PickupDelivery } from "../delivery/PickupDelivery.js";
import { CountryList } from "../models/CountryList.js";

export class ConsoleUI {
    private rl = readline.createInterface({ input, output });
    private orderService = new OrderService();

    public async start(): Promise<void> {
        await this.orderService.init();

        console.log("=== Система управления доставкой ===");
        console.log(`Данные успешно загружены из хранилища.`);

        let running = true;

        while (running) {
            clearConsole();

            console.log(`
\nГлавное меню:
1. Создать клиента
2. Создать курьера
3. Создать новый заказ
4. Назначить курьера на заказ
5. Изменить статус заказа
6. Показать все заказы
7. Удалить заказ
0. Выход
            `)

            const choice = await this.rl.question("\nВыберите пункт меню: ");

            try {
                switch (choice.trim()) {
                    case "1":
                        await this.createClientFlow();
                        break;
                    case "2":
                        await this.createCourierFlow();
                        break;
```

```
                    case "3":
                        await this.createOrderFlow();
                        break;
                    case "4":
                        await this.assignCourierFlow();
                        break;
                    case "5":
                        await this.changeOrderStatusFlow();
                        break;
                    case "6":
                        this.showAllOrders();
                        await this.rl.question("\nНажмите Enter, чтобы
вернуться в меню...");
                        break;
                    case "7":
                        await this.deleteOrderFlow();
                        break;
                    case "0":
                        running = false;
                        console.log("Выход из программы. До свидания!");
                        break;
                    default:
                        console.log("Неверный выбор. Попробуйте снова.");
                }
            } catch (error: any) {
                console.log(`\n[Ошибка]: ${error.message}`);
                await this.rl.question("\nНажмите Enter, чтобы
продолжить...");
            }
        }

        this.rl.close();
    }

    private async createClientFlow(): Promise<void> {
        const name = await this.rl.question("Введите имя клиента: ");
        const phone = await this.rl.question("Введите номер телефона клиента:
");
        const client = await this.orderService.createClient(name, phone);
        console.log(`Клиент "${client.name}" успешно создан и сохранен!`);
        await this.rl.question("\nНажмите Enter, чтобы продолжить...");
    }

    private async createCourierFlow(): Promise<void> {
        const name = await this.rl.question("Введите имя курьера: ");
        const phone = await this.rl.question("Введите номер телефона курьера:
");
        const courier = await this.orderService.createCourier(name, phone);
        console.log(`Курьер "${courier.name}" успешно создан и сохранен!`);
```

```
        await this.rl.question("\nНажмите Enter, чтобы продолжить...");
    }

    private async createOrderFlow(): Promise<void> {
        const clients = this.orderService.getClients();
        if (clients.length === 0) {
            throw new Error("Сначала создайте хотя бы одного клиента!");
        }

        console.log("\nСписок клиентов:");
        clients.forEach((c, index) => console.log(`${index + 1}. ${c.name}, ${
c.phone}`));
        const clientIndexStr = await this.rl.question("Выберите номер
клиента: ");
        const clientIndex = parseInt(clientIndexStr) - 1;

        console.log("\nВыберите страну доставки:");
        console.log("1. Россия");
        console.log("2. Казахстан");
        console.log("3. Беларусь");
        const countryChoice = await this.rl.question("Ваш выбор (1-3): ");

        let country: CountryList;
        switch (countryChoice.trim()) {
            case "1":
                country = CountryList.RUS;
                break;
            case "2":
                country = CountryList.KAZ;
                break;
            case "3":
                country = CountryList.BLR;
                break;
            default:
                throw new Error("Неверный выбор страны доставки!");
        }

        const address = await this.rl.question("Введите адрес доставки: ");

        const items: OrderItem[] = [];
        let addingItems = true;
        while (addingItems) {
            const prodName = await this.rl.question("Введите название товара
(или оставьте пустым для завершения): ");
            if (!prodName.trim()) break;

            const priceStr = await this.rl.question("Введите цену за единицу
($): ");
            const price = parseFloat(priceStr);
```

```
            if (isNaN(price) || price <= 0) throw new Error("Цена должна быть
положительным числом!");

            const weightStr = await this.rl.question("Введите вес товара за
единицу (грамм): ");
            const weight = parseFloat(weightStr);
            if (isNaN(weight) || weight <= 0) throw new Error("Вес должен
быть положительным числом!");

            const qtyStr = await this.rl.question("Введите количество: ");
            const quantity = parseInt(qtyStr);
            if (isNaN(quantity) || quantity <= 0) throw new Error("Количество
должно быть целым положительным числом!");

            items.push(new OrderItem(prodName.trim(), price, weight,
quantity));
            console.log("Товар добавлен в заказ.");

            const more = await this.rl.question("Добавить еще товар? (y/n):
");
            if (more.trim().toLowerCase() !== 'y') {
                addingItems = false;
            }
        }

        if (items.length === 0) {
            throw new Error("В заказе должен быть хотя бы один товар!");
        }

        console.log("\nВыберите способ доставки:");
        console.log("1. Стандартная");
        console.log("2. Экспресс");
        console.log("3. Самовывоз");
        const deliveryChoice = await this.rl.question("Ваш выбор (1-3): ");

        let deliveryMethod;
        switch (deliveryChoice.trim()) {
            case "1":
                deliveryMethod = new StandartDelivery();
                break;
            case "2":
                deliveryMethod = new ExpressDelivery();
                break;
            case "3":
                deliveryMethod = new PickupDelivery();
                break;
            default:
                throw new Error("Неверный выбор способа доставки!");
        }
```

```
        const order = await this.orderService.createOrder(clientIndex,
address, country, items, deliveryMethod);
        const itemsSum = order.getItemsCost();
        const deliveryCost = deliveryMethod.calculateCost(country,
order.getItemsWeight());

        console.log(`\nЗаказ №${order.id} успешно создан! Стоимость доставки:
$${deliveryCost}. Итого: $${itemsSum + deliveryCost}`);
        await this.rl.question("\nНажмите Enter, чтобы продолжить...");
    }

    private async assignCourierFlow(): Promise<void> {
        const orders = this.orderService.getOrders();
        const couriers = this.orderService.getCouriers();

        if (orders.length === 0) throw new Error("Нет доступных заказов.");
        if (couriers.length === 0) throw new Error("Нет зарегистрированных
курьеров.");

        // Фильтруем только свободных курьеров
        const availableCouriers = couriers.filter(c => !c.getIsBusy());
        if (availableCouriers.length === 0) {
            throw new Error("В данный момент нет свободных курьеров (все
заняты).");
        }

        console.log("\nСписок заказов:");
        orders.forEach(o => console.log(`ID: ${o.id} | Клиент: $
{o.client.name} | Номер: ${o.client.phone} | Статус: ${o.getStatus()}`));
        const orderIdStr = await this.rl.question("Введите ID заказа: ");
        const orderId = parseInt(orderIdStr);

        console.log("\nСписок свободных курьеров:");
        availableCouriers.forEach((c, index) => {
            const originalIndex = couriers.indexOf(c); // находим реальный
индекс в общем массиве сервиса
            console.log(`${index + 1}. ${c.name} | Тел: ${c.phone} (Индекс в
системе: ${originalIndex + 1})`);
        });

        const courierChoiceStr = await this.rl.question("Выберите номер
свободного курьера из списка: ");
        const choiceIndex = parseInt(courierChoiceStr) - 1;

        if (isNaN(choiceIndex) || !availableCouriers[choiceIndex]) {
            throw new Error("Неверный выбор курьера!");
        }

        const realCourierIndex =
couriers.indexOf(availableCouriers[choiceIndex]);
```

```
        await this.orderService.assignCourier(orderId, realCourierIndex);
        console.log(`Курьер успешно назначен на заказ №${orderId}!`);
        await this.rl.question("\nНажмите Enter, чтобы продолжить...");
    }

    private async changeOrderStatusFlow(): Promise<void> {
        const orders = this.orderService.getOrders();
        if (orders.length === 0) throw new Error("Нет доступных заказов.");

        console.log("\nСписок заказов:");
        orders.forEach(o => console.log(`ID: ${o.id} | Статус: $
{o.getStatus()}`));
        const orderIdStr = await this.rl.question("Введите ID заказа: ");
        const orderId = parseInt(orderIdStr);

        console.log("\nДоступные статусы:");
        const statuses = Object.values(OrderStatus);
        statuses.forEach((status, index) => console.log(`${index + 1}. $
{status}`));

        const statusIndexStr = await this.rl.question("Выберите новый номер
статуса: ");
        const statusIndex = parseInt(statusIndexStr) - 1;

        if (isNaN(statusIndex) || !statuses[statusIndex]) {
            throw new Error("Неверный статус!");
        }

        await this.orderService.changeOrderStatus(orderId,
statuses[statusIndex]);
        console.log(`Статус заказа №${orderId} успешно изменен!`);
        await this.rl.question("\nНажмите Enter, чтобы продолжить...");
    }

    private showAllOrders(): void {
        const orders = this.orderService.getOrders();
        if (orders.length === 0) {
            console.log("\nСписок заказов пуст.");
            return;
        }

        console.log("\n=== Список всех заказов ===");
        orders.forEach(o => {
            const itemsSum = o.getItemsCost();
            const itemsWeight = o.getItemsWeight();
            const country = o.country;
            const delivery = o.getDelivery();
            const deliveryCost = delivery ? delivery.calculateCost(country,
itemsWeight) : 0;
```

```
            const deliveryDays = delivery ? delivery.calculateDays(country,
itemsWeight) : 0;

            console.log(`-----------------------------------`);
            console.log(`Заказ №${o.id}`);
            console.log(`Страна доставки: ${country}`);
            console.log(`Клиент: ${o.client.name} | Номер: ${o.client.phone}
`);
            console.log(`Адрес: ${o.address}`);
            console.log(`Статус: ${o.getStatus()}`);
            console.log(`Товары:`);
            o.items.forEach(item => console.log(`  - ${item.productName} x$
{item.quantity} (${item.price}$/шт) = $${item.getTotalPrice()}`));
            console.log(`Общий вес товаров: ${itemsWeight} г`);
            console.log(`Сумма за товары: $${itemsSum}`);
            console.log(`Стоимость доставки: $${deliveryCost}`);
            console.log(`Итого к оплате: $${itemsSum + deliveryCost}`);
            console.log(`Срок доставки: ${deliveryDays} дн.`);
            console.log(`Курьер: ${o.getCourier() ? o.getCourier()!.name :
'Не назначен'} | Номер: ${o.getCourier() ? o.getCourier()!.phone : 'Не
назначен'}`);
        });
        console.log(`-----------------------------------`);
    }

    private async deleteOrderFlow(): Promise<void> {
        const orders = this.orderService.getOrders();

        if (orders.length === 0) {
            throw new Error("Нет доступных заказов для удаления.");
        }

        console.log("\n=== Удаление заказа ===");
        orders.forEach(o => console.log(`ID: ${o.id} | Клиент: $
{o.client.name} | Номер: ${o.client.phone} | Статус: ${o.getStatus()}`));

        const idChoiceStr = await this.rl.question("Введите ID заказа для
удаления: ");
        const idChoice = parseInt(idChoiceStr);

        if (isNaN(idChoice)) {
            throw new Error("ID заказа должен быть числом!");
        }

        await this.orderService.deleteOrder(idChoice)

        console.log(`\nЗаказ №${idChoice} успешно удален!`);
        await this.rl.question("\nНажмите Enter, чтобы продолжить...");
    }
}
```

```
export const clearConsole = (): Promise<void> => {
    return new Promise((resolve) => {
        if (!output.isTTY) {
            resolve();
            return;
        }
        output.write("\x1b[H\x1b[2J", () => resolve());
    });
}
```

5. Выводы

В ходе выполнения работы была успешно спроектирована, разработана и протестирована консольная система управления доставкой, реализованная на языке TypeScript с применением принципов объектно-ориентированного программирования (ООП) и стандартов чистой архитектуры.

В процессе реализации были достигнуты следующие ключевые результаты:

- Архитектурное разделение ответственности: Проект успешно разделен на изолированные логические слои (модели данных, полиморфные стратегии, сервисный слой бизнес-логики и пользовательский интерфейс), что гарантирует высокую поддерживаемость, расширяемость и тестируемость кода.
- Применение паттернов проектирования: Успешно внедрен паттерн «Стратегия» для вычисления стоимости и сроков различных видов доставки (стандартная, экспресс, самовывоз) через единый интерфейс `DeliveryMethodInterface`, что позволяет добавлять новые типы доставки без изменения существующего кода (соблюдение принципа открытости/закрытости OCP).
- Персистентность данных: Реализован асинхронный сервис хранения (`StorageService`), обеспечивающий автоматическую сериализацию и сохранение состояния всей системы в локальный файл `database.json`, а также восстановление данных при старте программы.

- Разработанное приложение полностью выполняет поставленные задачи, демонстрирует глубокое понимание принципов SOLID и ООП, а также может служить готовой программной основой для дальнейшего масштабирования — например, интеграции с реляционной базой данных или переноса логики на веб-фреймворк.

Весь исходный код был загружен на GitHub - https://github.com/Drakoshaaa/Delivery-app